

1. Arquitetura do DefendGraph

A arquitetura proposta foi concebida de modo a refletir a estrutura clássica de um sistema especialista. A interface gráfica do protótipo foi projetada para espelhar essa base conceitual, organizando o fluxo operacional de maneira estritamente sequencial em quatro estágios (*Stages*), os quais são subdivididos em passos específicos (*Steps*).

Na Tabela 1, apresenta-se o mapeamento do arcabouço tecnológico em relação aos blocos funcionais teóricos que lhe servem de fundamento:

Tabela 1. Mapeamento dos componentes de um sistema especialista (SE) no protótipo.

Componente SE	Implementação no protótipo	Tecnologias
Base de Conhecimento	A ontologia MITRE D3FEND é processada via as bibliotecas Python <i>rdflib</i> , <i>Owlready2</i> e <i>OWL-RL</i>	d3fend.owl (MITRE D3FEND), <i>rdflib</i> , <i>Owlready2</i> e <i>OWL-RL</i>
Memória de Trabalho	O fato primário é um arquivo JSON que representa um alerta nativo do Wazuh. Resultados intermediários persistidos por etapa.	Alerta Wazuh (JSON) e arquivos de estado (.json, .rdf, .owl, .html, .mmd e .md)
Mecanismo de Inferência	A consistência lógica é verificada pelo <i>reasoner</i> Pellet, via <i>Owlready2</i> . As relações implícitas são materializadas pelo OWL 2 RL, via <i>OWL-RL</i> . As consultas ao grafo seguem <i>Competency Questions</i> (CQs) definidas em SPARQL e executadas com <i>rdflib</i> .	Pellet (<i>Owlready2</i>), OWL 2 RL <i>OWL-RL</i> , SPARQL (<i>rdflib</i>)
Módulo de Explicação	Converte as deduções em narrativas estruturadas ao usuário, conforme <i>templates</i> predefinidos: texto em Markdown (.md) e diagramas em Mermaid (.mmd).	Markdown (.md) e diagramas Mermaid (.mmd).
Interface ao Usuário	Gerencia entradas, resultados, explicações e notificações ao usuário.	<i>Streamlit</i> , <i>Cytoscape</i> , <i>Mermaid</i>

2. Descrição dos Estágios (*Stages*) e Passos (*Steps*)

Com o objetivo de conciliar flexibilidade com rigor metodológico, a execução das etapas do processo é parametrizada, sempre que viável, por meio de arquivos de configuração independentes (*templates*). Destaca-se, adicionalmente, que cada passo da arquitetura opera com entradas e saídas materializadas em formatos de arquivo padronizados. A geração desses arquivos a cada transição de estado não apenas modulariza o sistema, como também viabiliza a auditoria completa de todo o ciclo de inferência.

A Figura 1 apresenta a arquitetura e o fluxo de execução dos estágios (*Stages*) e de seus respectivos passos (*Steps*).

A Figura 2 apresenta a tela inicial do sistema especialista semântico para fins ilustrativos.

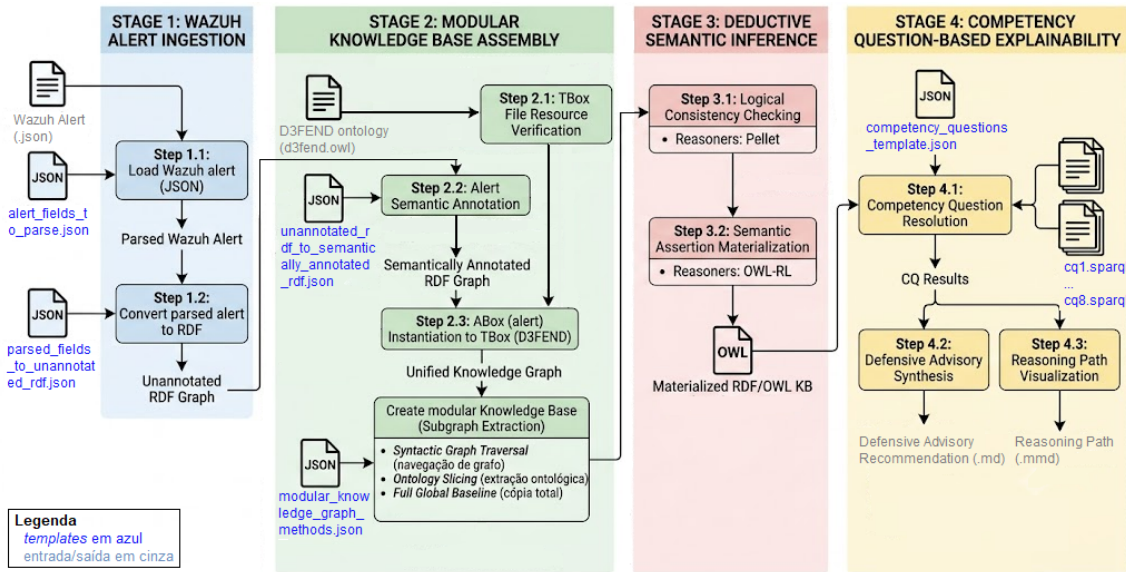


Figura 1. Arquitetura e fluxo de execução dos estágios (Stages) e passos (Steps)

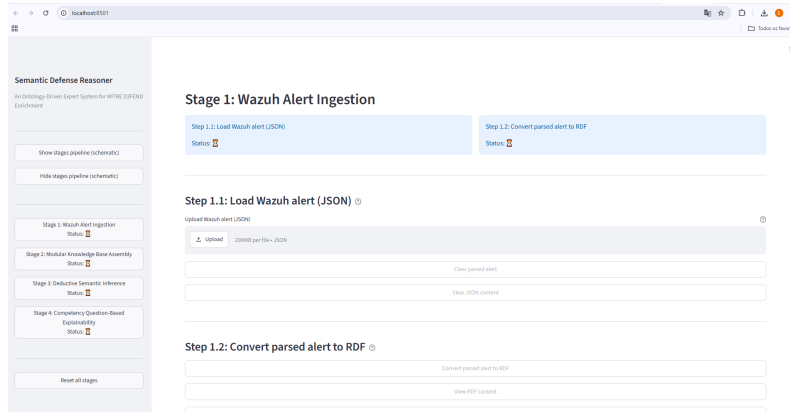


Figura 2. Tela inicial do DefendGraph

2.1. Stage 1: Wazuh Alert Ingestion

Este estágio corresponde à ingestão do alerta nativo gerado pela ferramenta SIEM Wazuh. No *Step 1.1: Load Wazuh alert (JSON)*, o usuário seleciona um arquivo no formato .JSON correspondente a um único alerta. O sistema realiza o parsing dos campos com base no template `alert_fields_to_parse.json`. Em seguida, o *Step 1.2: Convert parsed alert to RDF* executa a conversão estrutural dos dados para o formato RDF, de acordo com o template `parsed_fields_to_unannotated_rdf.json`.

Essa etapa de conversão estrutural prepara os dados para a vinculação semântica formal à ontologia D3FEND no estágio subsequente.

2.2. Stage 2: Modular Knowledge Base Assembly

O segundo estágio realiza a transição do dado puramente sintático para a representação semântica formal. A construção da Base de Conhecimento ocorre pela integração da ontologia de referência (*TBox*) com os dados instanciados do alerta (*ABox*).

O processo inicia-se no *Step 2.1: TBox File Resource Verification*, no qual é selecionado o arquivo correspondente à ontologia MITRE D3FEND em formato RDF/OWL para a verificação de sua existência, acessibilidade e integridade estrutural.

No *Step 2.2: Alert Semantic Annotation*, o grafo RDF resultante da extração sintática do alerta Wazuh é anotado semanticamente. O processo emprega *namespaces* e o vocabulário de domínio da D3FEND para assegurar a compatibilidade ontológica, mapeando formalmente os dados brutos em instâncias conceituais da ontologia. Esse procedimento opera conforme os parâmetros especificados no template `unannotated_rdf_to_semantically_annotated_rdf.json`.

Já o *Step 2.3: ABox (alert) Instantiation to TBox (D3FEND)* consolida as asserções geradas nos dois passos anteriores ao vincular logicamente a *ABox* (o evento do alerta) à *TBox*. Na sequência, o grafo unificado é serializado em um arquivo formal RDF/OWL.

Posteriormente, o *Create modular Knowledge Base (Subgraph Extraction)* provê uma camada de filtragem algorítmica que opera sobre a base de conhecimento unificada e não raciocinada, a fim de extrair um subgrafo com base em parâmetros de configuração predefinidos no template `modular_knowledge_graph_methods.json`. Três métodos são disponibilizados. O método *Syntactic Graph Traversal* realiza uma extração estrutural fundamentada na navegação do grafo RDF a partir do nó inicial do alerta, expandindo-se pelas conexões até uma profundidade limite configurável. Em contraste, o *Ontology Slicing* aplica uma extração orientada à ontologia, concentrando-se em capturar recursos específicos da D3FEND, bem como suas relações e hierarquias de classe associadas a um nó de partida. Por fim, o *Full Global Baseline* atua como método de grupo de controle, ao abdicar de qualquer processo de filtragem ou modularização, preservando e copiando a base de conhecimento global em sua totalidade.

2.3. Stage 3: Deductive Semantic Inference

Este estágio consubstancia o mecanismo de inferência do sistema especialista, mediante a aplicação de um raciocinador em Lógica de Descrição para verificar a consistência lógica da base de conhecimento e, uma vez considerada válida, computar seu fechamento dedutivo por meio da materialização de asserções semânticas implícitas.

No *Step 3.1: Logical Consistency Checking*, o *reasoner* Pellet avalia o grafo de conhecimento para assegurar que as asserções do alerta (*ABox*) não introduziram inconsistências lógicas em relação aos axiomas da ontologia de referência D3FEND (*TBox*). Uma vez atestada a consistência, o *Step 3.2: Semantic Assertion Materialization* computa o fechamento dedutivo da ontologia por meio do *reasoner* OWL-RL. Este processo culmina na geração de um arquivo formal RDF/OWL, no qual todas as asserções semânticas implicitamente inferidas são materializadas como declarações explícitas.

2.4. Stage 4: Competency Question-Based Explainability

O quarto estágio materializa a camada explicativa do sistema especialista, dedicada à resolução de questões de competência predefinidas sobre a base de conhecimento materializada e à transformação de seus resultados em explicações e recomendações defensivas.

O fluxo inicia-se com o *Step 4.1: Competency Question Resolution*, no qual um conjunto de *Competency Questions* parametrizadas em SPARQL interroga a base de co-

nhecimento para consultar o contexto do alerta, associações semânticas, caminhos de raciocínio e artefatos defensivos segundo a ontologia D3FEND.

A partir dessas respostas, o *Step 4.2: Defensive Advisory Synthesis* traduz os resultados das questões de competência em uma recomendação defensiva estruturada, combinando explicações, rastreabilidade do raciocínio, evidências de suporte e ações defensivas sugeridas. Por fim, o *Step 4.3: Reasoning Path Visualization* apresenta de forma gráfica o encadeamento de regras e asserções que conduziram o sistema àquela conclusão.

As *Competency Questions* são configuradas no template `competency_questions_template.json`, o qual referencia arquivos externos no formato `.sparql` que contêm as consultas propriamente ditas. Manter as consultas em arquivos separados favorece a legibilidade.